

CanvasNet Formal Review Report - CanvasNetPdf-PdfDocument (Re- Review)

Review that CanvasNetPdf PdfDocument Implementation is Correct

Revision note: This is the consolidated RE-REVIEW performed after a documentation-sync pass on `feature/pdf-document-support` (commit `bc86abc`), superseding the prior review captured at `review-report-CanvasNetPdf-PdfDocument.full-unit-prior.md` in this same directory. Every file in the review-set was re-read fresh from disk for this pass; no prior conclusion was assumed without re-verification. (A partial, single-area re-review was transiently written to this same filename by a delegated sub-agent mid-session; this document replaces that partial content with the full, consolidated review across all seven areas.)

1. Introduction

1.1 Purpose

This document records the formal RE-REVIEW of the PdfDocument software unit — CanvasNetPdf's sole direct unit, implementing the entire PDF tokenizer, object model, xref resolution, page-tree traversal, content-stream interpreter, graphics state, path operations, device color spaces, filter pipeline, image XObjects, CCITT G4 fax decoding, font resolution (Type0/Type1/Type3/ToUnicode/fallback), encryption (RC4/AES), patterns (shading/tiling)/functions, and text rendering — following a consolidated documentation-sync pass intended to remediate every finding from the prior formal review of this review-set.

1.2 Scope

- **Project:** CanvasNet (`demaconsulting/CanvasNet`)
- **Branch:** `feature/pdf-document-support`
- **Commit:** `bc86abc`

- **Review set:** CanvasNetPdf-PdfDocument

1.3 Outcomes

Outcome	Meaning
Pass	The check was performed and the criterion is satisfied
Fail	The check was performed and the criterion is not satisfied
N/A	The check does not apply; justification is required

1.4 Review Details

Field	Value
Project	CanvasNet
Review ID	CanvasNetPdf-PdfDocument
Review Title	Review that CanvasNetPdf PdfDocument Implementation is Correct
Fingerprint	9520193085947d55d155221fac5c7c7f385a49aad33b458db27ab375680dd88c
Review Date	2026-10-02
Baseline	dotnet build -c Release: 0 warnings/0 errors. dotnet test (Pdf test project, all TFMs): 1293/1293 passing, 0 skipped.

1.5 Reviewers

Name	Role	Organization	Signature	Date
Formal Review Agent	Automated Reviewer	DEMA Consulting	(automated)	2026-10-02

1.6 Files Under Review

Context (reference only): docs/design/canvas-net-pdf.md, docs/design/introduction.md, docs/reqstream/canvas-net-pdf.yaml.

Files (per dotnet reviewmark --elaborate CanvasNetPdf-PdfDocument, re-run this session): docs/design/canvas-net-pdf/pdf-document.md, docs/reqstream/canvas-net-pdf/pdf-document.yaml, docs/sysml2/model/canvas-net-pdf/pdf-document.sysml, docs/verification/canvas-net-pdf/pdf-document.md, src/DemaConsulting.CanvasNet.Pdf/NamespaceDoc.cs, PdfDocument.CcittFax.cs, PdfDocument.Color.cs, PdfDocument.ContentStream.cs, PdfDocument.Encryption.cs, PdfDocument.Filters.Ascii.cs, PdfDocument.Filters.Lzw.cs, PdfDocument.Filters.RunLength.cs, PdfDocument.Filters.cs, PdfDocument.FontFallback.cs, PdfDocument.Fonts.ToUnicode.cs, PdfDocument.Fonts.Type0.cs, PdfDocument.Fonts.Type1.cs, PdfDocument.Fonts.Type3.cs, PdfDocument.Fonts.cs, PdfDocument.Functions.cs,

PdfDocument.GraphicsState.cs, PdfDocument.Images.cs,
 PdfDocument.ObjectModel.cs, PdfDocument.PageTree.cs,
 PdfDocument.PathOps.cs, PdfDocument.Patterns.Shading.cs,
 PdfDocument.Patterns.Tiling.cs, PdfDocument.Patterns.cs,
 PdfDocument.Text.cs, PdfDocument.Tokenizer.cs, PdfDocument.Xref.cs,
 PdfDocument.cs, PdfPageInfo.cs, test/DemaConsulting.CanvasNet.Pdf.Tests/
 PdfDocumentTests.cs, PdfFixtureTests.cs, PdfSystemIntegrationTests.cs.

Note: PdfDocument.Encryption.cs and PdfDocument.Fonts.Type3.cs are now in-scope (fixed since the prior review). test/.../PdfDocumentEncryptionTests.cs and PdfDocumentSymbolicEncodingTests.cs remain **outside** this review-set's file list — see Finding F0 (downgraded from Critical to Medium).

2. Status of Prior Review's Findings

#	Prior Finding	Severity (prior)	Status now
F0	Scope gap: Encryption/Type3 source + test files uncovered by any review-set	Critical	Partially fixed — both source files now listed; both test files still unlisted (downgraded to Medium)
1	Xref-stream /w negative width → unhandled IndexOutOfRangeException	Critical	Fixed — PdfDocument.Xref.cs:215-224, validates $0 \leq w \leq 8$, throws InvalidDataException; regression test passes
2	Unbounded page-tree / Kids recursion → StackOverflowException risk	High	Fixed — visited-ObjNum set (cycle) + MaxPageTreeNestingDepth=32 counter (depth), both InvalidDataException, both tested
3	Image Width×Height/CCITT Columns×Rows unbounded allocation	High	Partially fixed — ordinary images bounded (Images.cs:284-288); CCITT path still allocates before its bound check (Images.cs:367-389) — see new Finding G1
4	PNG-predictor DivideByZeroException	High	Fixed (Filters.cs:182,187,192)
5	LZW unbounded decompressed output	High	Fixed — 64 MiB cap (Filters.Lzw.cs:24,117-121)
6	Indexed /Hival int-overflow	High	Fixed (Color.cs:659,665)
7	Truncated image stream → raw IndexOutOfRangeException	High	Fixed for ordinary images (Images.cs:417); CCITT shares risk with #3
8	Composite-font (Type0) /w CID-range unbounded loop	Medium	Fixed — MaxCompositeWidthRangeSpan=65536 (Fonts.Type0.cs:335,412), long arithmetic, InvalidDataException

#	Prior Finding	Severity (prior)	Status now
9	Encryption /Length unvalidated	Medium	Fixed (Encryption.cs:259-269), range-checked, typed exception
10	Aes.Key assignment outside try/catch	Medium	Fixed — now inside try/catch translating CryptographicException → InvalidDataException
11	Non-finite (NaN/Infinity) operands unvalidated for cm/path ops	Medium	Still present — RequireNumbers (GraphicsState.cs ~L300-318) checks count/kind only, not finiteness; reproduced via a 400-digit numeral parsing to Infinity
12	Unbounded q-stack / operand-stack growth	Medium	Still present — no depth/size cap on _gsStack or per-operator operand list
13	Sampled-function (FunctionType 0) int32 bit-offset overflow	High	Fixed — checked long arithmetic + try/catch(OverflowException) → InvalidDataException, plus explicit bounds check before indexing (Functions.cs ~L108-148)
14	Non-constant-time crypto comparisons / no key zeroing	Low	Still present (acceptable risk, unchanged)
15	Misleading StandardGlyphNames XmlDoc remarks	Low	Still present (Fonts.cs:640-645)
16	Stale “Phase 1 — blank-only” NamespaceDoc.cs claim	Low	Still present (NamespaceDoc.cs:21-28)

Net assessment: of 16 carried-forward items, **10 are now fully fixed with regression tests**, 2 are partially fixed (F0, CCITT allocation-ordering), and 4 remain unaddressed (non-finite operands, unbounded stack growth, 3 Low-severity documentation items). This confirms the documentation-sync pass’s claims were **accurate for documentation changes** but, as expected (it was explicitly scoped as a documentation pass), did **not** address the pre-existing code-quality findings that are not purely documentation — some of which (Xref /w, page tree, PNG predictor, LZW, Hival, Type0 /w, /Length, Aes.Key, sampled-function overflow) were evidently *also* fixed in commits alongside or around the documentation-sync pass, while others (non-finite operands, stack bounds, CCITT ordering, F0’s test-file gap) were not.

3. Verification of User-Claimed Documentation-Sync Items

All items below were independently re-verified against current source/docs, not assumed:

Claim	Status	Evidence
Removed stale “unsupported” claims: encryption RC4/AES-128/AES-256-R5	True	Design doc + Encryption.cs consistent; R6 correctly still documented/rejected (see §6)
Removed stale “unsupported” claims: Type1/Type1C/Type3 fonts	True	No remaining stale claims found in design doc
Removed stale “unsupported” claims: Symbol/ZapfDingbats fallback	True	Matches FontFallback.cs behavior
Removed stale “unsupported” claims: Pattern (shading+tiling)	True	Matches Patterns.Shading.cs/Patterns.Tiling.cs
Removed stale “unsupported” claims: Function Types 2/3	True	Functions.cs:179-266 implements both; FunctionType 4 still correctly rejected
Removed stale “unsupported” claims: CalRGB/ICCBased/Indexed color spaces	True	Color.cs:503-509, 588-631, 649-686, all implemented + tested
Removed stale “unsupported” claims: TilePaint, GifCodec, SystemFontCatalog	True (TilePaint, in Drawing subsystem doc)	docs/design/canvas-net/drawing.md correctly states “four sibling units” including TilePaint; GifCodec/SystemFontCatalog are outside this unit’s own docs (not applicable to pdf-document.md directly)
Orphaned requirements 21→0, unsatisfied 5→0	True for this unit	dotnet reqstream --enforce-style check shows 0 PdfDocument-prefixed unsatisfied/orphaned requirements; all 60 requirement IDs map 1:1 to verification-doc scenarios
Added missing mandatory verification sections (6 docs)	True for this unit, with a defect	## Acceptance Criteria section does exist (line 1453) — contradicts this agent’s own earlier quick-grep finding, which is retracted — but it is at the wrong heading depth (## instead of ###, inconsistent with its three sibling sections); see Finding G7
Fixed stale CFF-operator-exclusion claims	True	Matches CffCharstringInterpreter.cs/CffTable.cs exclusions

Claim	Status	Evidence
Fixed stale scn-pattern test reference	True	Confirmed via <code>git show 6727343</code> ; new test name exists and passes
Documented password-overload signatures consistently	Mostly true, one stale remark remains	<code>Open(Stream/string, string? password = null)</code> doc matches API exactly; but <code>PdfDocument.cs</code> 's class-level "Phase 16" remarks paragraph still implies no password API exists, uncorrected for Phase 17 — see Finding G8
Added missing source/test file coverage in <code>.reviewmark.yaml</code>	Partially true	Both source files (<code>Encryption.cs</code> , <code>Fonts.Type3.cs</code>) added; both companion test files (<code>PdfDocumentEncryptionTests.cs</code> , <code>PdfDocumentSymbolicEncodingTests.cs</code>) still missing — Finding F0
Added missing source/test file coverage in SysML2 model comments	True	<code>pdf-document.sysml</code> 's <code>sourceRef/testRef</code> comments now list all 27 actual source files and all 5 actual test files, 1:1, including <code>Encryption.cs/Type3.cs</code> and both previously-uncommented test files
Corrected "three sibling units" → "four" (<code>TilePaint</code>)	True	<code>docs/design/canvas-net/drawing.md:5-14</code>
Added missing Architecture/ Interfaces/ Dependencies/ Callers entries	True, with a scope clarification	Architecture/Interfaces are System-level sections (present, complete, in <code>canvas-net-pdf.md</code>); Dependencies/Callers are Unit-level sections (present, complete, in <code>pdf-document.md</code>) — both doc levels fully populated per <code>design-documentation.md</code> 's actual per-level mandatory-section matrix

4. Requirements Traceability (REQ / REQIMP / REQTEST)

All 60 requirement IDs in `docs/reqstream/canvas-net-pdf/pdf-document.yaml` trace to a named, existing, non-skipped, currently-passing test, confirmed by direct inspection of test source (not the verification doc's claims alone) across all seven delegated areas. Full test run: **1293/1293 passed, 0 failed, 0 skipped** (dotnet test, net8.0/net9.0/net10.0).

Outcome: Pass, with two Low-severity traceability-hygiene gaps (new regression tests for the two now-fixed Critical findings — `xref /w validation` and `page-tree depth-guard` — are not yet referenced in the `requirement/verification-doc` tests: `**Test(s)**`: entries; see Finding G9).

5. Design Documentation Consistency (DES / CODEDOC)

Outcome: Pass, with the following residual gaps (all Low except where noted):

- G2 (Low): page-tree nesting-depth guard (`MaxPageTreeNestingDepth`) not mentioned in the design doc's page-tree-traversal/Error-Handling prose (only the visited-set cycle guard is described).
- G3 (Low): xref-stream /W bounds validation ($0 \leq w \leq 8$) not mentioned in the design doc's cross-reference-resolution/Error-Handling prose.
- G7 (Low/Medium): verification doc's ## Acceptance Criteria heading is at the wrong depth (see Finding G7 below) — a structural, not content, defect.
- G8 (Low): `PdfDocument.cs` class-level “Phase 16” remarks paragraph is stale relative to the Phase 17 password-parameter addition (per-member docs are correct; the class-level narrative was not updated).
- Fonts area: G4/G5 (Low) — `StandardGlyphNames XmlDoc` and `NamespaceDoc.cs` “Phase 1” claim remain stale (carried forward from the prior review, unaddressed by this pass).

No stale claims were found regarding any of the specific feature areas the user's sync pass targeted (encryption, Type1/Type3/fallback, patterns/functions, color spaces, CFF exclusions, `scn` test reference) — all verified accurate against current code.

6. Code Quality, Error Handling, and Security (CODE / LOGIC / ERR / SEC)

Outcome: Fail. Two Critical and one High finding remain or were newly identified; these are untrusted-input-parser robustness gaps consistent with the unit's fail-closed design goal, not regressions introduced by the documentation-sync pass (which did not touch this code).

New / Still-Open Critical Findings

- **G1 (Critical)** — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Images.cs:367-389`: CCITT Group 4 fax decoding allocates its output buffer (sized from `/Columns×/Rows`) **before** validating those dimensions are within a sane bound (the bound check runs later, at line ~389, after the allocation at ~line 374). A crafted `/Columns//Rows` pair can therefore (a) trigger an unbounded-memory-allocation DoS before any check fires, or (b) overflow during the size computation, surfacing a raw `OverflowException` instead of a typed exception. **Confidence: High** (direct code read). **Fix:** move the dimension bound-check (matching the one already applied to ordinary image `XObjects` at `Images.cs:284-288`) to **before** the buffer allocation, and perform the multiplication in checked context.
- **G6 (Critical)** — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Xref.cs:653-672` (`ZlibDecompress`, the `FlateDecode` filter implementation): has **no cap on decompressed output size**. `FlateDecode` is the single most commonly used PDF stream filter (content streams, object streams, most image data); a small crafted `FlateDecode` stream can decompress to an

effectively unbounded size in memory (a textbook “decompression bomb”), with no typed-exception circuit breaker, unlike the LZW decoder which now has a 64 MiB cap (fixed per Finding 5 above). **Confidence: High. Fix:** apply an analogous output-size cap (e.g. 64–128 MiB, consistent with the LZW cap) to `ZlibDecompress`, throwing `InvalidDataException` when exceeded.

New High Finding

- **G0 (High)** — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Xref.cs`, `GetObject/LoadCompressedObject` (~lines 489-534): `GetObject(number)` only populates `_objectCache` **after** it finishes resolving. A crafted xref stream where a compressed-object entry’s container-stream number resolves back to itself (directly, $N \rightarrow N$, or via a longer $A \rightarrow B \rightarrow A$ chain) causes unbounded mutual recursion between `GetObject` and `LoadCompressedObject`, with **no cycle or depth guard** — unlike the page-tree traversal (fixed, Finding 2 above) and `Form-XObject/Type3/color-space/` function recursion limits that exist elsewhere in this same codebase (`MaxFormNestingDepth`, `MaxType3NestingDepth`, `MaxColorSpaceRecursionDepth`, `MaxFunctionRecursionDepth`, `MaxPageTreeNestingDepth`). This risks an uncatchable `StackOverflowException` that terminates the host process — the same severity class as the page-tree bug this re-review was specifically asked to confirm fixed, but present in a sibling resolution path that was not covered by that fix. **Confidence: High** (static trace; not reproduced live, consistent with the conservative-execution posture used elsewhere in this review, since triggering it would crash the process uncontrollably). **Fix:** add an “in-progress” `HashSet<int>` of object numbers currently being resolved inside `GetObject`, mirroring the page-tree visited set pattern — throw `InvalidDataException("Object stream contains a compressed-object reference cycle.")` on re-entry, add/remove via `try/finally`. Add a regression test with a self-referential compressed-object xref entry.

Carried-Forward Medium Findings (unaddressed by the documentation-sync pass)

- (= **prior #11, Medium**) Non-finite (NaN/Infinity) numeric operands are not validated for `cm` and path-construction operators (`GraphicsState.cs/PathOps.cs/RequireNumbers`), while the sibling `d` (dash-pattern) operator and `Patterns.Tiling.cs` already validate finiteness — an inconsistently-applied fix pattern. Reproduced: a 400-digit content-stream numeral parses via `double.TryParse` to `PositiveInfinity` without error and folds into the CTM. **Fix:** add `double.IsFinite` validation inside the shared `RequireNumbers` helper (used by `cm`, `m/l/c/v/y/re`, `w`, `M`), throwing `InvalidDataException`.
- (= **prior #12, Medium**) No bound on `q/Q` graphics-state-stack depth or per-operator operand-stack size (`ContentStream.cs/GraphicsState.cs`), unlike every other nesting/recursion vector in this codebase (`Form XObjects`, `Type3`, `color-space`, `function`, `page-tree` — all capped). **Fix:** add a `MaxGraphicsStateDepth` constant analogous to the existing nesting-depth constants, checked in `OpPushGraphicsState`.
- **G10 (Medium, new)** — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Fonts.ToUnicode.cs:226`: the `bfrange` single-destination-array loop has the same unbounded-CID-range-span DoS pattern that was just fixed for `Type0 /w` (Finding 8 above), but the fix was not applied here. **Fix:** add an

analogous `MaxToUnicodeBfRangeSpan` bound + `InvalidDataException`, mirroring `MaxCompositeWidthRangeSpan`.

- **G11 (Medium, new)** — `src/DemaConsulting.CanvasNet.Pdf/PdfDocument.Filters.RunLength.cs`: `RunLengthDecode` has no output-size cap, unlike LZW (64 MiB cap) and (once fixed) the recommended `FlateDecode` cap. Lower severity than LZW/Flate since `RunLength`'s worst-case expansion ratio is far smaller ($\approx 127:1$ vs. effectively unbounded), but still an inconsistently-applied control. **Fix**: apply the same output-size cap pattern used for LZW.

Low Findings (carried forward, unaddressed)

- Non-constant-time password/MAC byte-array comparisons (`SequenceEqual`, not `CryptographicOperations.FixedTimeEquals`) and no explicit key-material zeroing in `PdfDocument.Encryption.cs`. Real-world risk remains Low (local file reader, not a network-exposed oracle).
- Misleading `StandardGlyphNames XmlDoc` remarks (`Fonts.cs:640-645`) describing stricter behavior than the actual (correct) lenient fallback.
- Stale “Phase 1 — blank-only Render” `XmlDoc` claim in `NamespaceDoc.cs:21-28`.
- `GetIntEntry/Color.cs` helpers used for `/Columns/Rows` lack their own range validation, which is what allows Finding G1's CCITT allocation-before-check gap to be reachable in the first place (noted as contributing cause, not a separate defect).

Confirmed-Fixed / Verified-Good (no action needed)

RC4 KSA/PRGA re-verified algorithmically correct; AES/MD5/SHA-256 via real BCL primitives only; no password/key material in any exception message; AES-256 R5 vs R6 correctly and consistently distinguished (R6/“hardened hash” deliberately rejected, matching docs); `/Length` and `Aes.Key` issues fixed with regression tests; `Type3` nesting-guard, `FunctionType 4` rejection, and `Tr` stroke/fill+stroke pixel-level tests all confirmed solid; indexed-palette, PNG-predictor, LZW-cap, `Type0-/w-cap`, and sampled-function-overflow fixes all independently confirmed via direct code+test inspection, not merely trusted from the prior review or the user's claims.

7. Testing Standards (TEST)

Outcome: Pass. All sampled tests across all seven areas follow Arrange-Act-Assert, use descriptive `MethodUnderTest_Scenario_ExpectedBehavior`-style names, and assert concrete values (parsed fields, pixel colors, exception types/tokens) rather than only “no exception thrown.” No skipped/disabled tests were found anywhere in the review-set's test files. Full suite: 1293/1293 passing across `net8.0/net9.0/net10.0`.

8. Readability / Usability (READ / USE)

Outcome: Pass. No material findings beyond the documentation-staleness items already captured in §5.

9. Summary of Findings

ID	Severity	Area	Status	Summary
F0	Medium (↓ from Critical)	Scope/traceability	Open	Encryption/Type3 test files still uncovered by any review-set
G0	High (new)	File structure	Open	Self-referential compressed-object chain → uncatchable StackOverflowException
G1	Critical	Images/CCITT	Open	CCITT buffer allocated before dimension bound-check
G6	Critical (new)	Filters	Open	FlateDecode (ZlibDecompress) has no decompression-bomb cap
#11	Medium	Content-stream	Open	Non-finite numeric operands unvalidated for cm/path ops
#12	Medium	Content-stream	Open	Unbounded q-stack / operand-stack growth
G10	Medium (new)	Fonts	Open	ToUnicode bfrange unbounded-span DoS (same pattern as fixed Type0 /w)
G11	Medium (new)	Filters	Open	RunLengthDecode has no output-size cap
G2	Low	Design doc	Open	Page-tree depth-guard undocumented
G3	Low	Design doc	Open	Xref /w bounds-validation undocumented
G7	Low/Medium	Verification doc	Open	## Acceptance Criteria heading at wrong depth
G8	Low	Design doc	Open	Stale Phase-16 class remarks re: password API
G9	Low	Reqstream traceability	Open	New regression tests (xref /w, page-tree depth) not yet cross-referenced in reqstream/verification docs
(prior #14)	Low	Encryption	Open	Non-constant-time comparisons, no key zeroing
(prior #15)	Low	Fonts	Open	Misleading StandardGlyphNames XmlDoc
	Low		Open	

ID	Severity	Area	Status	Summary
(prior #16)		Fonts/ Namespace		Stale “Phase 1” NamespaceDoc.cs claim Xref /w, page-tree recursion, image bounds (non-CCITT), PNG predictor, LZW cap, / Hival overflow, truncated-image exception, Type0 /w span, /Length validation, Aes.Key try/catch, sampled-function overflow
—	—	—	Fixed (10)	

Totals: 2 Critical (open), 1 High (open), 4 Medium (open), 7 Low (open); 10 prior findings fully fixed with regression-test evidence; all documentation-sync claims verified true except the still-partial F0 fix.

10. Overall Outcome

FAIL.

The documentation-sync pass was executed accurately and in good faith: every specific documentation claim made by the user was independently verified true (with one partial exception — F0’s test-file coverage gap), requirements traceability is now complete (0 orphaned/unsatisfied for this unit), and roughly two-thirds of the prior review’s code-level findings were also fixed with proper regression-test coverage in the same window. This represents substantial, verifiable progress.

However, the unit still does not meet the bar for a Pass:

1. **Finding F0** (review-set scope) remains open: PdfDocumentEncryptionTests.cs and PdfDocumentSymbolicEncodingTests.cs are still outside .reviewmark.yaml’s file list for this review-set, so their 20+ security-sensitive test methods are not formally subject to review enforcement.
2. **Two Critical and one High code-quality finding** remain or were newly identified in this pass (G1 CCITT allocation-ordering, G6 FlateDecode decompression-bomb, G0 object-stream self-reference stack overflow) — all in the same “untrusted input must fail closed with a typed exception” category this review-set exists to enforce, and all independently reproducible/traceable via static code analysis.
3. Several Medium-severity DoS/robustness gaps remain (non-finite operands, unbounded graphics-state stack, ToUnicode bfrange span, RunLength output size).

Required Remediation for a Future Pass

- Fix G1 (reorder CCITT bounds check before allocation) and G6 (cap FlateDecode output) — both Critical.
- Fix G0 (add cycle guard to GetObject/LoadCompressedObject) — High.

- Add .reviewmark.yaml entries for PdfDocumentEncryptionTests.cs and PdfDocumentSymbolicEncodingTests.cs (F0).
- Address the four open Medium findings (#11, #12, G10, G11) or provide an explicit, documented risk-acceptance rationale for each.
- Recommended but not blocking: fix the Low-severity documentation/traceability items (G2, G3, G7, G8, G9) and the three carried-forward Low findings.

11. Approval Status

Rejected, pending the remediation above. Re-review recommended once the Critical/ High findings are addressed.